

`./figs/solartower.jpg`

Optimal Wiring of Heliostats in Solar Tower Power Plants

Duc Thanh Tran

27 June 2018

Problem

- ▶ Consider a complete and undirected graph $G = (V, E)$ with $V := \{T\} \cup V_H$, where T is the designated solar tower and V_H is the set of heliostats.
- ▶ Goal: Find a subgraphs $H_{\text{data}}, H_P \subseteq G$ with minimal total costs that connects all heliostats with the solar tower.
- ▶ Constraints:
 - ▶ Maximum number of connected heliostats per cable is limited by $h_{\max} := 128$.
 - ▶ No intersecting edges.
 - ▶ Consider switch limitations and their costs.

Motivation

What factors need to be considered for the HWP?

Motivation

Local Control Units

Multi-port or Conductor?

Motivation

Local Control Units

Multi-port or Conductor?

Location

Australia vs South Africa

Motivation

Local Control Units

Multi-port or Conductor?

Location

Australia vs South Africa

Partitioning

Partition structure? How many partitions?

Motivation

Local Control Units

Multi-port or Conductor?

Location

Australia vs South Africa

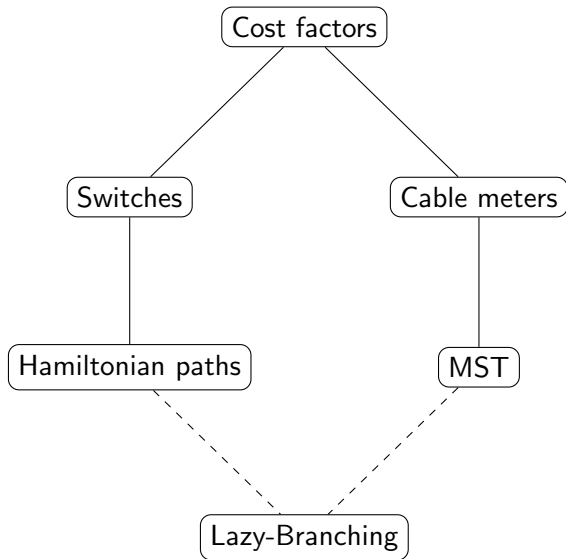
Partitioning

Partition structure? How many partitions?

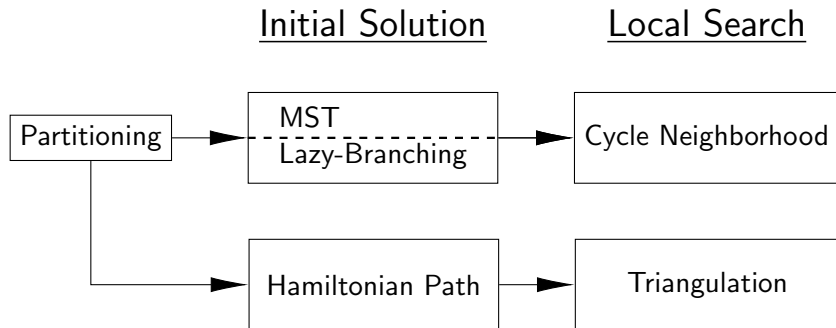
Planarity

How does the planarity constraint influence the algorithm?

Initial solutions



Our approach



Partitioning

Maximum number of connected heliostats per cable is limited by

$$h_{max} = 128.$$

- ▶ Partition heliostats beforehand: Calculate angles between vectors and the x-axis.
- ▶ Compute initial solutions on each partition independently.

Partitioning

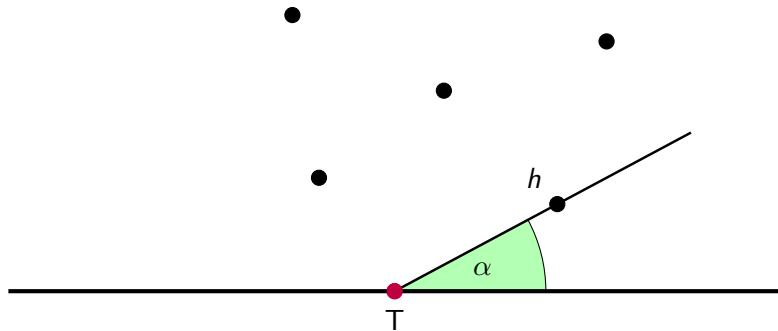


Figure: Compute angle between x-axis and $\text{line}(T, h)$, $h \in V_H$.

Partitioning

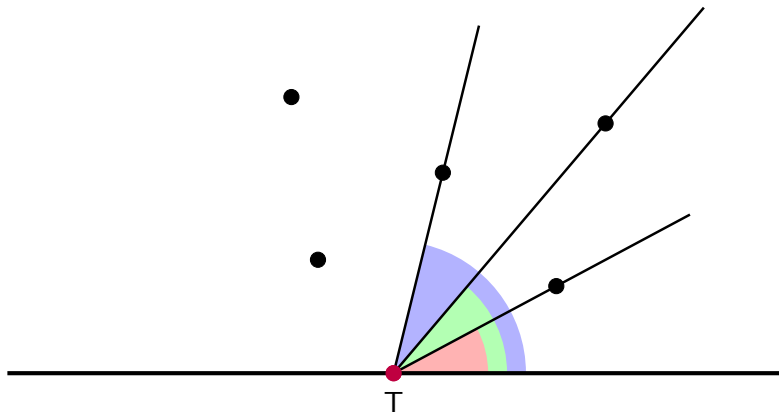


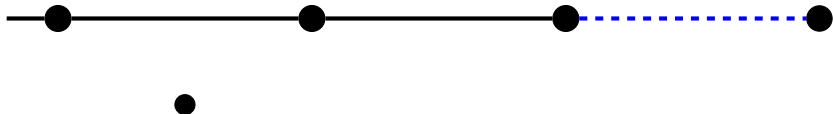
Figure: Sort heliostats by computed angles.

Heuristic: Hamiltonian Path

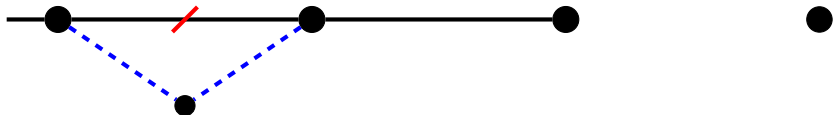
Initial Solution: Hamiltonian Path

i) Append

add edge - - -



ii) Insert



Local Search: Hamiltonian Path – Triangulation

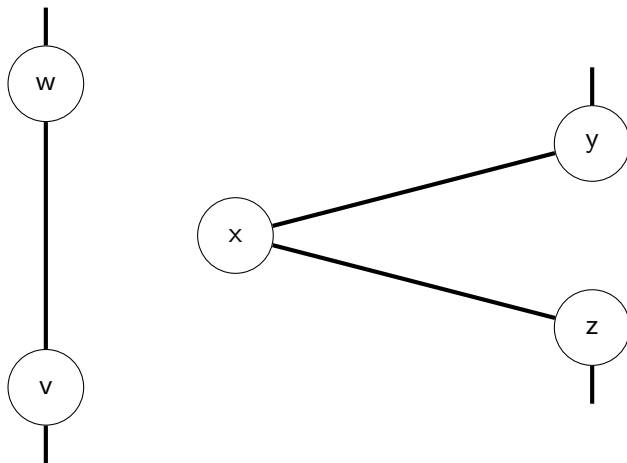


Figure: Starting point.

Local Search: Hamiltonian Path – Triangulation

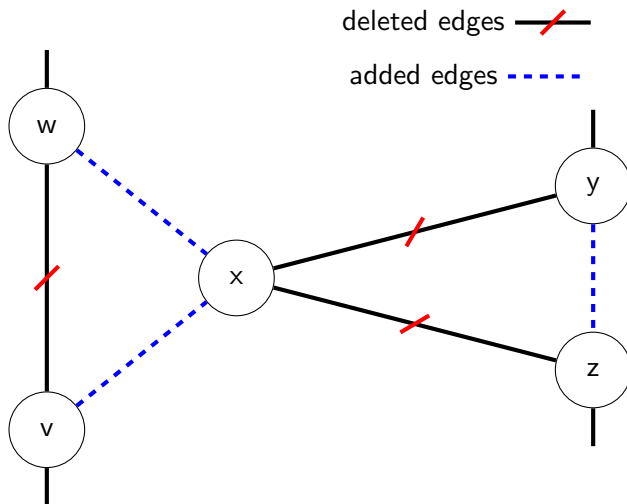


Figure: Check costs of added and deleted edges.

Local Search: Hamiltonian Path – Triangulation

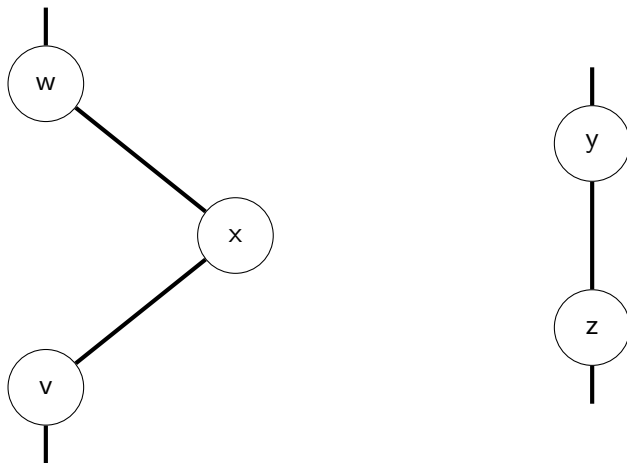
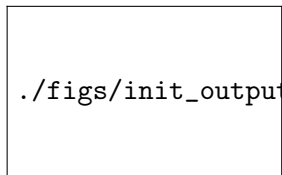
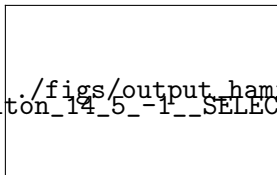


Figure: Added edges have lower costs than deleted ones.

Triangulation Neighborhood Improvement



(a) Initial solution.



(b) After triangulation improvement.

The image is a large, empty rectangular frame with a thin black border. In the center of the frame, the text `./figs/init_output_hamilton_14_5_-1.png` is displayed in a monospaced font. This text likely refers to a file that contains the visualization of the initial Hamiltonian path, which is not visible in this image.

`./figs/init_output_hamilton_14_5_-1.png`

Figure: Initial Hamiltonian path. Total costs: 311,381.07€.



`./figs/output_hamilton_14_5_-1.png`

Figure: After local search for Hamiltonian paths. Total costs:
301.177.01€

Heuristic: Minimum Spanning Tree (MST)

Initial Solution: MST

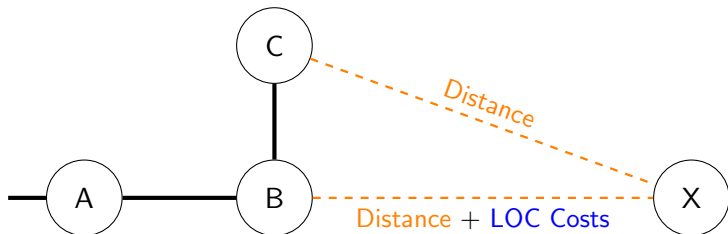


Figure: Include switch installation into edge costs.

Local Search: Cycle Neighborhood

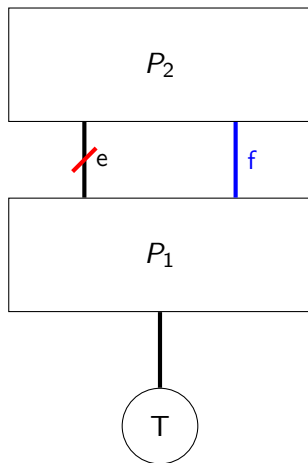


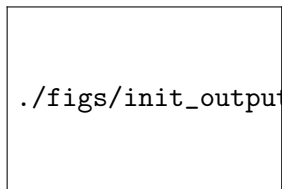
Figure: Consider the cycle neighborhood, that is, delete edge e and reconnect the resulting partitions P_1 and P_2 by adding an edge f with $c(f) < c(e)$.

The figure area is mostly empty, with the text './figs/init_output_mst_14_6.png' centered. This text likely refers to a file containing a diagram of the initial Minimum Spanning Tree (MST) solution.

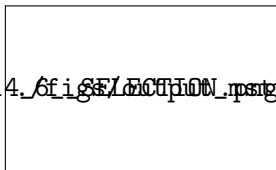
`./figs/init_output_mst_14_6.png`

Figure: Initial MST solution. Total costs: 317,311.75€.

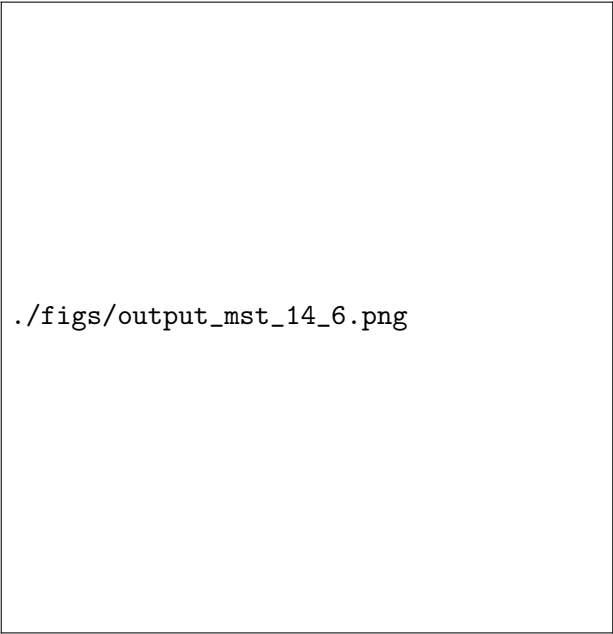
Cycle Neighborhood Improvement



(a) Initial solution.



(b) After cycle improvement.



`./figs/output_mst_14_6.png`

Figure: After local search for spanning trees. Total costs: 313,385.10€.

Heuristic: Lazy-Branching

Lazy-Branching (= MST + Hamiltonian path)

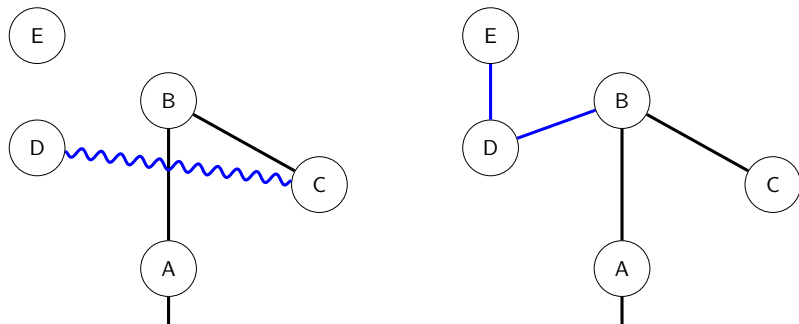


Figure: Edge CD intersects edge AB , therefore we install a switch at B and branch out. We continue extending the path from D on.



`./figs/init_output_lazy-branching_14_6.png`

Figure: Initial Lazy-Branching solution. Total costs: 321,802.35€.



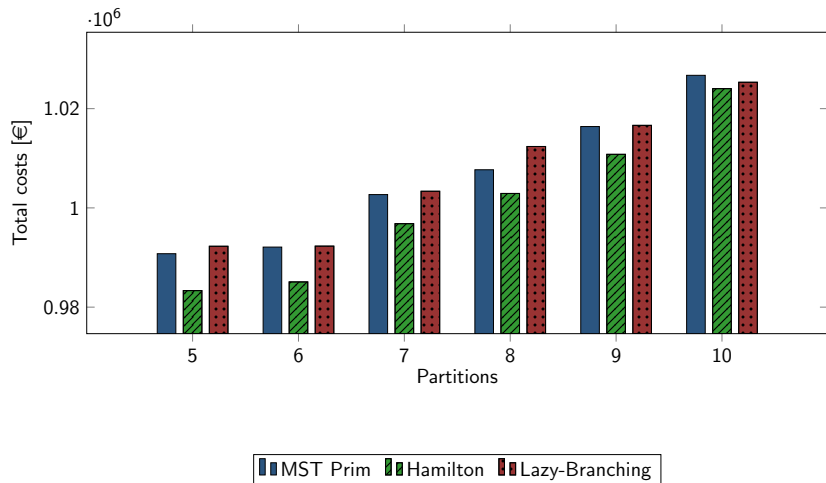
`./figs/output_lazy-branching_14_6.png`

Figure: After local search with cycle neighborhood. Total costs:

314 415 306

Evaluations

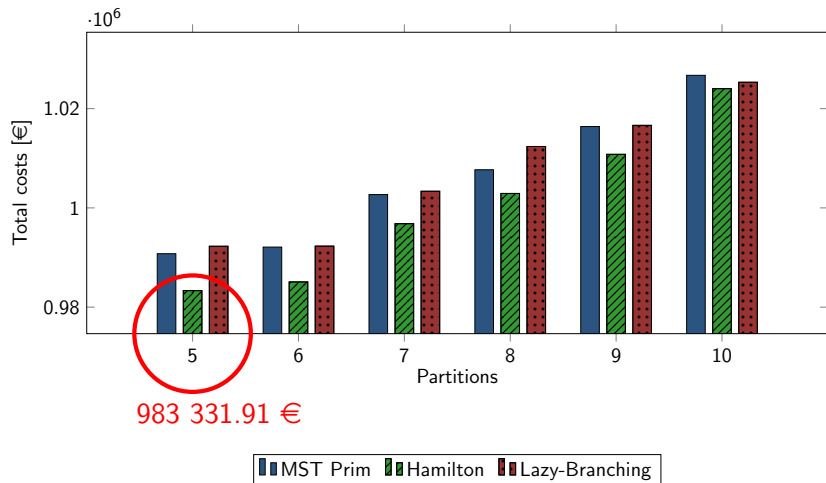
Australia



Local search improvement:

MST: 1.8 % , Hamilton: 3.9 % , Lazy-Branching: 4.1 %

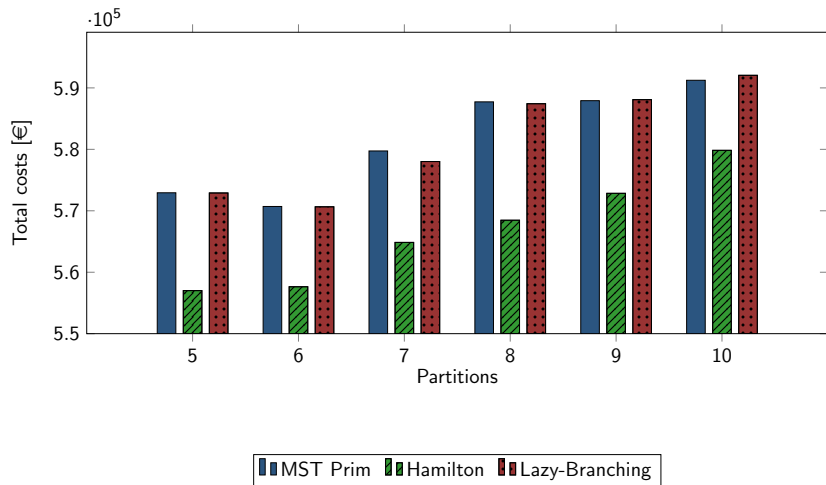
Australia



Local search improvement:

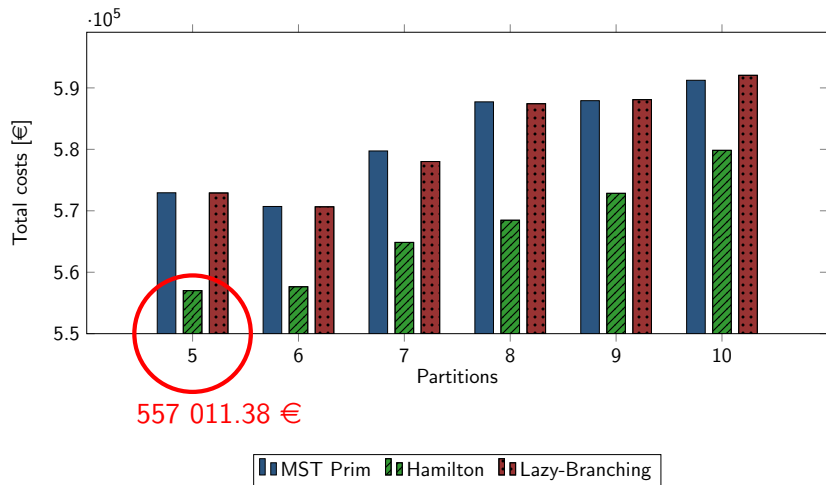
MST: 1.8 % , Hamilton: 3.9 % , Lazy-Branching: 4.1 %

Spain



Local search improvement:

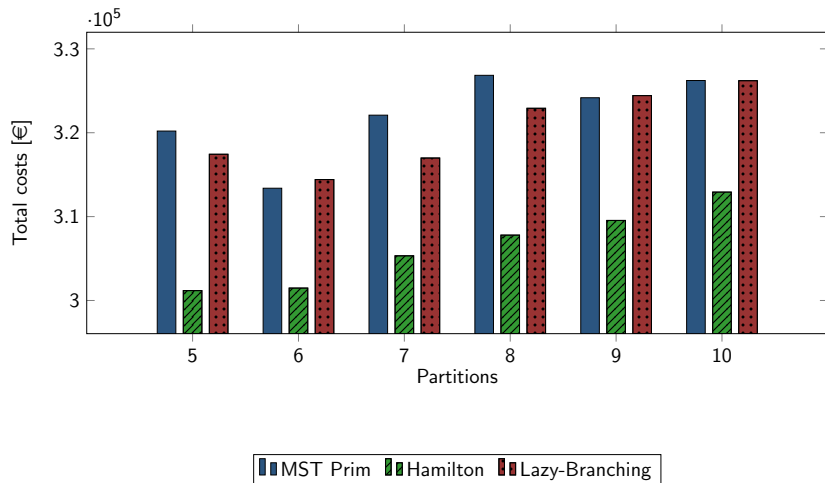
MST: 1.3 %, Hamilton: 3.9 %, Lazy-Branching: 2.1 %



Local search improvement:

MST: 1.3 %, Hamilton: 3.9 %, Lazy-Branching: 2.1 %

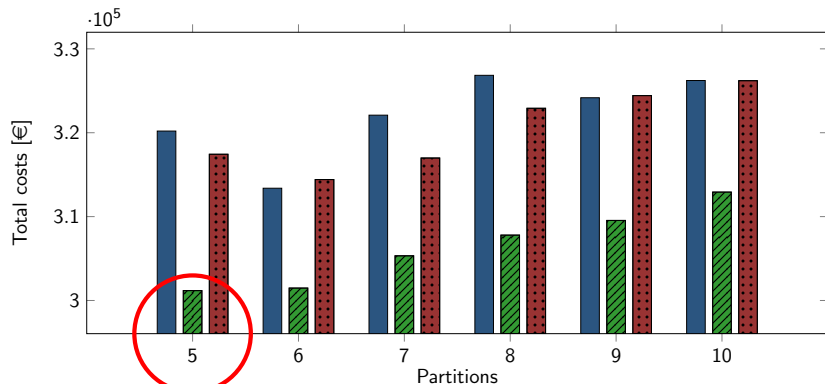
South Africa/UAE



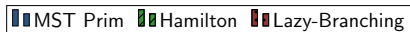
Local search improvement:

MST: 1.2 %, Hamilton: 3.9 %, Lazy-Branching: 2.3 %

South Africa/UAE



301 179.16 €



Local search improvement:

MST: 1.2 %, Hamilton: 3.9 %, Lazy-Branching: 2.3 %

Comparison for 5 Partitions before Local Search

Table: Total costs and cable lengths for 5 partitions. We boldfaced the best initial values for each country.

Algorithm	Australia		Spain		UAE/RSA	
	Cost[€]	Length[m]	Cost[€]	Length[m]	Cost[€]	Length[m]
Hamiltonian Path	1,022K	17.78K	578.1K	17.78K	311.4K	17.78K
MST	1,008K	16.70K	585.4K	17.05K	326.1K	17.63K
Lazy-Branching	1,033K	17.61K	593.3K	17.61K	328.9K	17.64K

Comparison for 5 Partitions after Local Search

Table: Total costs and cable lengths for 5 partitions. We boldfaced the best locally optimal solution values for each country. The difference to the initial solutions are shown in percentages.

Algorithm	Australia		Spain		UAE/RSA	
	Cost[€]	Length[m]	Cost[€]	Length[m]	Cost[€]	Length[m]
Hamiltonian Path	983.3K (-3.9%)	17.04K (-4.3%)	557.3K (-3.7%)	17.04K (-4.3%)	301.0K (-3.5%)	17.06K (-4.2%)
MST	990.8K (-1.8%)	16.82K (+0.7%)	572.9K (-2.2%)	17.17K (+0.7%)	320.2K (-1.8%)	17.71K (+0.5%)
Lazy-Branching	992.3K (-4.1%)	16.88K (-4.3%)	572.9K (-3.6%)	17.17K (-2.6%)	317.4K (-3.6%)	17.82K (+1%)

Location Comparison

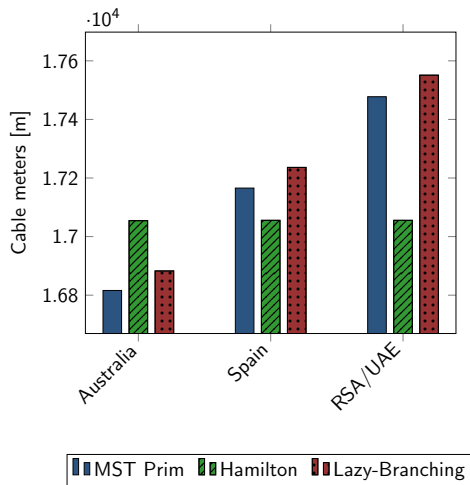


Figure: Cable lengths for 6 partitions.

Conclusion

Unanimous result

Conclusion

Unanimous result



**Hamiltonian path with 5 tours
delivers best results**

`./figs/recipes.png`

Thank you for your attention!

Appendix: Algorithms

Initial Solution: Hamiltonian Path

Algorithm 1 Nearest-insertion heuristic for Hamiltonian path construction.

```
1: function NEAREST-INSERTION-HAMILTONIAN( $G = (V, E), c$ )
2:    $Seq := [T]$  ▷ Ordered sequence of vertices
3:   while  $|Seq| < |V|$  do
4:      $(v, index) := \text{BEST-CANDIDATE}(Seq, V, c)$ 
5:      $Seq[index] := v$ 
6:   end while
7:   construct corresponding edges from  $Seq$ , yielding  $E'$ 
8:   return  $(V, E')$ 
9: end function
```

Local Search: Triangulation for Hamiltonian Path

Algorithm 2 Local Search: Triangulation

```
1: function LOCAL-SEARCH-TRIANGULATION( $G = (V, E), c$ )
2:   while not stable do                                ▷ No edge can be improved
3:     for  $vw \in E$  do
4:       IMPROVE-EDGE( $vw, G, c$ )
5:     end for
6:   end while
7:   return  $G = (V, E)$ 
8: end function
9:
10: function IMPROVE-EDGE( $vw, G, c$ )
11:    $(xy, xz) := \text{GET-BEST-TRIANGULATION}(vw, G)$ 
12:   ( $xz$  may be NULL)
13:   if HAS-NEGATIVE-COSTS( $vw, x, G, c$ ) then
14:      $E := E \setminus \{vw, xy, xz\}$ 
15:      $E := E \cup \{vx, wx\} \cup \{yz\}$                 ▷ last edge set can be empty
16:   end if
17:   return
18: end function
```

Initial Solution: Minimum Spanning Tree

Algorithm 3 Prim's Algorithm with additional intersecting-constraint.

```
1: function MST-PRIM( $G = (V, E), c$ )
2:    $E' := \emptyset$ 
3:   while  $|E'| < |V|$  do
4:      $F := \text{SORTED-CUT-EDGES}(E', c)$        $\triangleright$  Consider edges
        $T_w \in E$  if  $E' = \emptyset$ 
5:     for  $e \in F$  do
6:       if NOT-INTERSECTING( $e, E'$ ) then
7:          $E' := E' \cup e$ 
8:       go to 3
9:     end if
10:  end for
11:  end while
12:  return  $(V, E')$ 
13: end function
```

Initial Solution: Lazy-branching

Algorithm 4 Hybrid algorithm that computes a spanning tree by extending paths greedily as possible. Otherwise we branch off and install switches.

```
1: function LAZY-BRANCHING( $G = (V, E)$ ,  $c$ )
2:    $last := T$ , mark  $T$  as visited                                ▷ Start from solar tower
3:    $E' := \emptyset$ 
4:   while  $|E'| < |V|$  do
5:     select unvisited vertex  $w \in V$  such that  $c_e$  is minimized for  $e :=$ 
        $\{last, w\} \in E$ 
6:     if NOT-INTERSECTING( $e, E'$ ) then                                ▷ Path
7:        $E' := E' \cup e$ 
8:        $last := w$ , mark  $w$  as visited
9:     else                                                            ▷ Branching
10:      select visited vertex  $v \in V$  such that  $c_f$  is minimized for  $f :=$ 
         $\{v, w\} \in E$ 
11:       $E' := E' \cup f$ 
12:       $last := w$ , mark  $w$  as visited
13:    end if
14:  end while
15:  return  $(V, E')$ 
16: end function
```

Local Search: Cycle Neighborhood

Algorithm 5 Local search: Cycle Neighborhood for Spanning Trees

```
1: function LOCAL-SEARCH-CYCLE-NEIGHBORHOOD( $G = (V, E), c$ )
2:   while not stable do                                     ▷ No edge can be improved
3:     for  $e \in E$  do
4:       IMPROVE-EDGE( $e, G, c$ )
5:     end for
6:   end while
7:   return  $G = (V, E)$ 
8: end function
9:
10: function IMPROVE-EDGE( $e, G, c$ )
11:    $E := E \setminus \{e\} \Rightarrow$  obtain disjoint partitions  $V_1, V_2 \subseteq V$ 
12:   for  $e \in$  SORTED-CUT-EDGES( $V_1, V_2, c$ ) do
13:     if NOT-INTERSECTING( $e, E$ ) then
14:        $E := E \cup \{e\}$ 
15:       return
16:     end if
17:   end for
18:    $E := E \cup \{e\}$                                        ▷ No improvement by replacing  $e$ 
19:   return
20: end function
```